

Informatics Institute of Technology
affiliated with
University of Westminster

Client Server Architectures

5COSC022W

Module Leader: Mr. Cassim Farook

**“Smart Campus” Sensor & Room
Management API**

Name: Warnakulasuriya Uddika Hasun Tisera

UOW Number: W2153004

IIT Number: 20240397

Report: Question Responses

Question 1.1

In your report, explain the default lifecycle of a JAX-RS Resource class. Is a new instance instantiated for every incoming request, or does the runtime treat it as a singleton? Elaborate on how this architectural decision impacts the way you manage and synchronize your in-memory data structures (maps/lists) to prevent data loss or race conditions.

Answer:

JAX-RS resource classes follow a **request-scoped lifecycle** by default. The JAX-RS runtime creates a brand new instance of each resource class, such as **RoomResource** or **SensorResource**, for every single incoming HTTP request. Once the request is processed and the response is sent, that instance is discarded and becomes eligible for garbage collection. This is the opposite of a singleton model where one shared instance would serve all requests for the lifetime of the application.

Because each request gets its own fresh resource instance, any data stored in instance variables would be destroyed the moment the request ends. Instance variables are therefore completely unsuitable for storing shared application data like rooms, sensors, or readings.

To address this, all data in this API lives in **static fields** inside the **DataStore** class:

```
public static Map<String, Room> rooms = Collections.synchronizedMap(new LinkedHashMap<>());

public static Map<String, Sensor> sensors = Collections.synchronizedMap(new
LinkedHashMap<>());

public static Map<String, List<SensorReading>> sensorReadings =
Collections.synchronizedMap(new LinkedHashMap<>());
```

Static fields belong to the class itself rather than to any instance, so they persist for the entire application lifetime regardless of how many resource instances come and go.

However, this introduces a concurrency challenge. Apache Tomcat processes multiple requests simultaneously on separate threads, meaning several threads can read from and write to these shared maps at the same time. A regular **HashMap** is not thread-safe; concurrent modifications can corrupt its internal state, causing infinite loops, lost updates, or **ConcurrentModificationException** errors.

Collections.synchronizedMap solves this by wrapping the **LinkedHashMap** with a single mutex lock, ensuring only one thread can access the map at a time. This prevents concurrent modification errors and data corruption, making it safe for use as shared in-memory state in a multi-threaded JAX-RS environment.

Question 1.2

Why is the provision of "Hypermedia" (links and navigation within responses) considered a hallmark of advanced RESTful design (HATEOAS)? How does this approach benefit client developers compared to static documentation?

Answer:

HATEOAS (Hypermedia as the Engine of Application State) is a REST architectural constraint where API responses include hyperlinks that guide clients to related resources and available next actions. Rather than requiring clients to memorise or hard-code URL patterns from external documentation, they discover and navigate the API dynamically by following links embedded in the responses themselves.

In this API, the discovery endpoint at **GET /api/v1/** returns links to all primary collections (Sample

```
{
  "links": {
    "rooms": "http://localhost:8080/Smart_Campus_API/api/v1/rooms",
    "sensors": "http://localhost:8080/Smart_Campus_API/api/v1/sensors"
  }
}
```

below):

Each room in the list response also includes its own **href** pointing directly to its detail endpoint. These links are generated dynamically using **uriInfo.getBaseUri()**, meaning they automatically reflect the correct host, port, and context path at runtime rather than being hardcoded strings.

Compared to static documentation, HATEOAS has a key advantage: the links in the response are always accurate because the server produces them at request time. Static documentation is written at a point in time and drifts out of sync as the API evolves. URLs change, endpoints are added, and documentation updates are forgotten. Clients built against stale documentation break without warning.

HATEOAS also reduces coupling. If a URL structure changes, clients that navigate by following named links such as "rooms" or "sensor_readings" adapt without code changes, because they are not depending on the literal path string. A client only needs to know one entry point, /api/v1/, and can discover everything else from the responses themselves. This makes both the client and the API more resilient to change over time.

Question 2.1

When returning a list of rooms, what are the implications of returning only IDs versus returning the full room objects? Consider network bandwidth and client-side processing.

Answer:

Returning only IDs forces every client to make a separate follow-up request for each room to retrieve any useful information such as the name or capacity. For a campus with thousands of rooms this creates the N+1 request problem: one list request spawns thousands of individual detail requests. This creates unnecessary load on the server and results in slow response times for the client, particularly on mobile networks.

Returning full room objects avoids the extra requests but transfers all fields including the sensorIds list for every room in every list response. Most clients, such as a UI displaying a room selection dropdown, only need the name and ID. Forcing all clients to download and parse complete objects wastes bandwidth and

increases memory consumption on the client, which is especially problematic on mobile or low-powered devices.

This API returns **summary objects** with only **id**, **name**, and **href**:

```
[
  {
    "id": "R101",
    "name": "Lab Room",
    "href": "http://localhost:8080/Smart_Campus_API/api/v1/rooms/R101"
  }
]
```

This strikes the right balance. It provides enough information for common use cases like displaying a list or populating a form, while keeping responses lightweight and fast. Clients that need full room details follow the href link to the detail endpoint. The embedded link also satisfies HATEOAS principles since the client does not need to construct the detail URL itself. This pattern scales well from small deployments to large ones without requiring changes to the API design.

Question 2.2

Is the DELETE operation idempotent in your implementation? Provide a detailed justification by describing what happens if a client mistakenly sends the exact same DELETE request for a room multiple times.

Answer:

Yes, the DELETE operation is idempotent in this implementation. An HTTP method is idempotent if making the same request multiple times produces the same **server-side state** as making it once. The key word is state, not response code.

When **DELETE /api/v1/rooms/R101** is called for the first time, the room is found in **DataStore.rooms**, the sensor check passes, the room is removed, and **204 No Content** is returned. If the same request is sent again, **DataStore.rooms.get("R101")** returns null because the room no longer exists. The null check triggers and **404 Not Found** is returned. Every subsequent call returns the same **404**.

The server state is identical after the first call and every call after it: room R101 does not exist. This satisfies idempotency even though the response code changes from 204 to 404. The desired end state, "this room does not exist", is achieved after the first call and remains unchanged by any subsequent calls.

This behaviour matters for reliability. Networks are unreliable and clients sometimes retry requests after a timeout, not knowing whether the original reached the server. Because DELETE is idempotent here, retrying is always safe. The worst outcome is a 404 response, which simply confirms the room was already gone. A non-idempotent delete that caused side effects on repeated calls, such as decrementing a counter or triggering cascading deletes again, would be dangerous in a retry scenario.

Question 3.1

We explicitly use the `@Consumes(MediaType.APPLICATION_JSON)` annotation on the POST method. Explain the technical consequences if a client attempts to send data in a different format, such as `text/plain` or `application/xml`. How does JAX-RS handle this mismatch?

Answer:

The `@Consumes(MediaType.APPLICATION_JSON)` annotation declares to the JAX-RS runtime that the annotated method will only accept requests whose **Content-Type** header is **application/json**. This is enforced during request routing before the resource method is ever called.

If a client sends a request with **Content-Type: text/plain** or **Content-Type: application/xml**, the JAX-RS runtime compares the incoming content type against the `@Consumes` declarations of all candidate methods for that path and HTTP verb. Finding no match, the framework automatically returns **415 Unsupported Media Type**. The `registerSensor` method never executes and no business logic runs.

The underlying mechanism is the **MessageBodyReader** system. JAX-RS uses registered readers to deserialize request bodies into Java objects. In this project, the Jackson library provides a **MessageBodyReader** for converting **application/json** to a **Sensor** instance. If the content type is something else, no suitable reader exists for that combination, and the request is rejected at the framework layer immediately.

This is beneficial for multiple reasons. Clients get a clear, standard error code that tells them exactly what went wrong without any ambiguity. The resource method is protected from receiving malformed or unexpected input. It also prevents content-type confusion attacks where a malicious client might attempt to send XML with XXE payloads or other format-specific exploits to an endpoint expecting JSON. The annotation acts as a declarative contract that the framework enforces automatically.

In this implementation, the **NotSupportedExceptionMapper** added in the Additional Implementations section intercepts this exception before it reaches the **GlobalExceptionHandler**, restoring the correct **415 Unsupported Media Type** response.

Question 3.2

You implemented this filtering using `@QueryParam`. Contrast this with an alternative design where the type is part of the URL path (e.g., `/api/v1/sensors/type/CO2`). Why is the query parameter approach generally considered superior for filtering and searching collections?

Answer:

Path parameters and query parameters serve fundamentally different semantic purposes. Path parameters identify **which specific resource** is being accessed. Query parameters describe **how to filter or shape** the results returned from a collection. Conflating the two leads to API designs that misrepresent the underlying domain model.

Using `/api/v1/sensors/type/CO2` as a path implies that **type/CO2** is a hierarchical sub-resource of **sensors**, suggesting a resource structure that does not exist. Sensor type is an attribute used to narrow a collection, not a child resource. This design also creates an awkward gap: a separate URL pattern is needed just to retrieve all sensors with no filter, leading to redundant routes and duplicated handler code.

The query parameter approach avoids all of these problems. **GET /api/v1/sensors** and **GET /api/v1/sensors?type=CO2** both use the same endpoint, the same method, and the same handler method.

The parameter is naturally optional, which is the correct behaviour for a filter. When the parameter is absent, the full collection is returned. When it is present, the collection is filtered. This is achieved with a single null check on the **type** parameter.

Multiple filters also compose cleanly with query parameters: **?type=CO2&status=ACTIVE** adds another filter with no changes to the URL structure or handler signature. Achieving the same with path parameters would require defining a separate **@GET** method with a different **@Path** for every possible filter combination, which grows exponentially as more attributes are added. Query parameters are the REST convention specifically designed for filtering, searching, and paginating collections, and keeping path segments reserved for resource identification produces cleaner, more intuitive, and more maintainable APIs.

Question 4.1

Discuss the architectural benefits of the Sub-Resource Locator pattern. How does delegating logic to separate classes help manage complexity in large APIs compared to defining every nested path in one massive controller class?

Answer:

The Sub-Resource Locator pattern allows a resource class to delegate handling of a sub-path to a separate, dedicated class. In this API, **SensorResource** delegates all reading operations to **SensorReadingResource**:

```
@Path("/{sensorId}/readings")
public SensorReadingResource getSensorReadingResource(@PathParam("sensorId") String
sensorId) {
    if (!DataStore.sensors.containsKey(sensorId)) {
        throw new LinkedResourceNotFoundException("Sensor with ID " + sensorId + " does
not exist");
    }
    return new SensorReadingResource(sensorId);
}
```

Without this pattern, all sensor and reading endpoints would live in one class. As the API grows with additional reading operations such as filtering by date range, pagination, or deleting individual readings, **SensorResource** becomes a Huge Class with hundreds of lines of mixed responsibilities. Finding, understanding, and modifying specific logic becomes difficult, and the risk of accidentally breaking unrelated endpoints increases.

By separating the classes, **SensorResource** focuses exclusively on sensor-level operations and **SensorReadingResource** focuses exclusively on reading-level operations. Each class has one clear responsibility. The sensor existence check is written exactly once in the locator method and applies to all reading operations automatically, rather than being repeated at the top of every individual method.

The **sensorId** is passed to **SensorReadingResource** via its constructor, giving every method in that class automatic access to the correct context without re-extracting it from path parameters. In a team environment, two developers can work on these classes simultaneously with no merge conflicts. In testing, **SensorReadingResource** can be instantiated and unit tested in complete isolation. These benefits scale significantly as the API grows in depth and the number of nested resources increases.

Question 5.1

Why is HTTP 422 often considered more semantically accurate than a standard 404 when the issue is a missing reference inside a valid JSON payload?

Answer:

404 Not Found communicates that the resource identified by the requested URL does not exist. When a client calls **POST /api/v1/sensors**, the URL **/api/v1/sensors** is valid and the endpoint exists. Returning **404** here would mislead the client into thinking it sent the request to the wrong URL, which is not the problem at all.

The real issue is inside the request body. The JSON is syntactically valid, all required fields are present, and the content type is correct. The problem is that the **roomId** value references a room that does not exist, which is a semantic violation of referential integrity rather than a routing or parsing failure.

422 Unprocessable Entity was defined (RFC 4918) precisely for this scenario. It signals that the server understood the request, parsed the JSON successfully, but cannot process the instructions because the data violates a business rule. This maps exactly to the case where a valid sensor payload references a non-existent room.

The distinction has practical importance for client developers. A **404** suggests the client should check its URL. A **422** suggests the client should check the data inside the request body, the **roomId** field in this case. This gives the client actionable, unambiguous information for debugging. It also allows client applications to handle data validation errors distinctly from routing errors in automated error handling, improving the overall developer experience of the API.

Question 5.2

From a cybersecurity standpoint, explain the risks associated with exposing internal Java stack traces to external API consumers. What specific information could an attacker gather from such a trace?

Answer:

Without the **GlobalExceptionHandler**, any unhandled exception would expose a raw Java stack trace in the API response. This is a serious information disclosure vulnerability that gives attackers a detailed map of the application internals.

A stack trace reveals the **internal package and class structure**, for example **io.github.epicer12.smart_campus_api.resources.SensorResource:45**. This allows an attacker to understand the application's architecture without source code access. Knowing exact class names, method names, and line numbers helps in crafting targeted attack payloads and reasoning about the application's internal flow.

Stack traces also expose the **technology stack and library versions**. Frames from Jersey, Tomcat, Jackson, or a database connector can be cross-referenced with CVE databases to find known vulnerabilities for those exact versions. If the trace reveals an outdated version of a library with a published exploit, the attacker can attempt it immediately.

Exception messages reveal **business logic weaknesses**. A message like **NullPointerException: cannot invoke String.length() because roomId is null** tells an attacker that sending null values for certain fields

may crash the application. This guides systematic fuzzing to discover further vulnerabilities and can facilitate denial of service by repeatedly triggering crashes.

The **GlobalExceptionHandler** addresses all of this by catching every **Throwable** and returning only a generic **500 Internal Server Error** message to the client, while the full exception details remain available in server-side logs for developers. Attackers receive no useful intelligence from the response, while the development team retains full visibility into what went wrong.

Question 5.3

Why is it advantageous to use JAX-RS filters for cross-cutting concerns like logging, rather than manually inserting `Logger.info()` statements inside every single resource method?

Answer:

Cross-cutting concerns are functionalities that apply across the entire application but belong to none of its individual components. Logging is the classic example. Implementing it manually by inserting log statements into every resource method creates several serious problems.

Every method across every resource class requires nearly identical log statements for the incoming request and outgoing response, violating the DRY principle and creating a large maintenance surface. When new endpoints are added, developers may forget to add logging, resulting in gaps in observability that are hard to detect and debug. If an exception is thrown before the response log statement is reached, that line never executes, so error responses frequently go unlogged. Any change to the log format, such as adding a timestamp, correlation ID, or request duration, requires editing every method in every resource class.

Manual logging also pollutes business logic. A **createRoom** method should contain only room creation logic. When it is surrounded by log statements, the actual intent of the code becomes harder to read.

The **LoggingFilter** solves all of these problems in one class registered once with **@Provider**:

```
@Provider
public class LoggingFilter implements ContainerRequestFilter, ContainerResponseFilter {

    @Override
    public void filter(ContainerRequestContext requestContext) throws IOException {
        LOGGER.info("Incoming Request: [" + requestContext.getMethod() + "] "
            + requestContext.getUriInfo().getRequestUri().toString());
    }

    @Override
    public void filter(ContainerRequestContext requestContext, ContainerResponseContext
responseContext) throws IOException {
        LOGGER.info("Outgoing Response: Status " + responseContext.getStatus());
    }
}
```

It intercepts every request and response across the entire API automatically, including requests that result in exceptions, because the response filter runs after the exception mapper has already converted the exception to a proper HTTP response. Every interaction is guaranteed to be logged with no gaps and no manual effort.

Future enhancements such as adding correlation IDs or measuring response time require changing only this one file. Resource methods stay clean and focused entirely on business logic. This is the Single Responsibility Principle in practice: the filter owns observability, the resource classes own domain operations.